

Automated Design of Multilevel Thresholding Segmentation: Single-Point vs. Multipoint

Thambo Nyathi^{a,*}, Mzoxolo Mbini^b

^a*Department of Computer Science, University of Pretoria, Pretoria, South Africa*

^b*Department of Computer Science, University of Pretoria, Pretoria, South Africa*

Abstract

Multilevel thresholding is a core image segmentation task, the computational cost of which increases exponentially with the number of thresholds. Meta-heuristic optimisers efficiently locate near-optimal threshold vectors. However, their performance depends on the hyperparameter settings. Automated algorithm design replaces manual tuning. However, for tight evaluation budgets, the relative merits of single-point and multipoint configuration strategies remain unknown.

This study provides the first systematic empirical comparison of these strategies for multilevel thresholding with constrained budgets. We held the base optimiser constant (Genetic Algorithm or Artificial Bee Colony) and applied four design paradigms: baseline defaults, search-enhanced iterated local search, single-point SA configuration, and multipoint evolutionary configurators (meta-GA and differential evolution). Ten variants were evaluated on 24 natural and remote sensing images across nine threshold levels using Kapur's entropy and Otsu's between-class variance, with total fitness evaluations strictly equalised across all algorithms at each threshold.

Under the tight outer-loop budgets used here, where multipoint configurators received 20% of the total evaluations for the hyperparameter search, no configurator outperformed well-chosen defaults. The outer loop consumed resources

*Corresponding author.

Email address: `t.nyathi@up.ac.za` (Thambo Nyathi)

without discovering superior configurations, leaving the inner search with both suboptimal hyperparameters and a 20% evaluation shortfall. This produced a consistent SSIM deficit of 0.16–0.20 relative to the baseline. Single-point ILS hybrids matched baseline quality at 83–92% lower runtime, making them the most efficient option under constrained budgets. These results demonstrate empirically that for multilevel thresholding under tight resource constraints, the budget split between the configuration and search layers is the dominant determinant of the outcome. Whether an adequately resourced multipoint configuration can surpass the baseline and what minimum outer allocation is required to recover the configuration cost remain open questions that motivate a systematic budget-scaling study.

Keywords: Multilevel thresholding, Automated algorithm design, Hyperparameter optimisation, Iterated local search, Differential evolution, Image segmentation

1. Introduction

Image segmentation partitions a digital image into meaningful regions based on the pixel intensity. Thresholding is one of the most widely adopted segmentation techniques owing to its conceptual simplicity and computational efficiency (Al-Amri et al., 2010). In bilevel thresholding, a single intensity value Th separates the foreground from the background. In multilevel thresholding (MLT), $k \geq 2$ thresholds divide the histogram into $k+1$ classes. The challenge is to select the threshold vector $\mathbf{t} = \{t_1, t_2, \dots, t_k\}$ that optimises a statistical objective, such as the between-class variance (Otsu et al., 1975) or entropy (Kapur et al., 1985). This is because the exhaustive search scales as $O(L^{k-1})$ where L is the number of grey levels. The MLT is NP-hard, and exact methods become impractical for $k > 4$.

Metaheuristic algorithms avoid the exponential cost of exhaustive search by guiding a stochastic search through threshold space. Genetic algorithms (GA) (Holland, 1992), particle swarm optimisation (Kennedy & Eberhart, 1995),

differential evolution (DE) (Storn & Price, 1997), and artificial bee colony (ABC) (Karaboga, 2005) have all been adapted to MLT with varying degrees of success. Despite this progress, two practical issues remain. First, standard metaheuristics require careful calibration of the population size, mutation rate, cooling schedule, and other hyperparameters. Poor choices lead to premature convergence and prohibitive runtimes. Second, the relationship between these hyperparameters and the resulting segmentation quality is non-stationary across images and threshold levels.

Automated algorithm design treats the hyperparameter vector of a target algorithm as a point in a configuration space and searches for a setting that maximises performance on a given problem distribution (Hutter et al., 2009). A fundamental divide in this field separates *single-point* configurators, which navigate the configuration space via a trajectory-based search, from *multipoint* configurators, which maintain a population of candidate configurations. Representative single-point approaches include ParamILS (Hutter et al., 2009) and simulated annealing-based tuners (Van Laarhoven & Aarts, 1987). Representative multipoint approaches include racing procedures (López-Ibáñez et al., 2016) and meta-genetic algorithms (Grefenstette, 1986). More recent surrogate-model methods (SMAC (Hutter et al., 2011), Hyperband, BOHB (Falkner et al., 2018)) have further expanded this toolkit. However, these approaches incur substantial overhead per configuration evaluation. When the inner fitness function is inexpensive to evaluate, as in MLT, where a single evaluation requires only a histogram sweep, the surrogate overhead can exceed the cost of the search itself. Therefore, derivative-free single-point and multipoint configurators remain relevant baselines for MLT.

A separate design choice is the method of conducting the inner threshold search. Beyond hyperparameter configuration, the search quality can be improved by augmenting the base algorithm with lightweight local search operators applied directly to the threshold vectors, such as iterated local search (ILS) (Lourengo et al., 2003) and simulated annealing (SA) (Van Laarhoven & Aarts, 1987). These *search enhancements* do not reconfigure the hyperparameters of

the base algorithm; instead, they extend or supplement the search itself. The distinction is important because a comparison between the outer-loop configuration and inner-loop search enhancement conflates two different interventions. This study was explicit about where each method was applied, and the experimental design was structured accordingly.

Surprisingly, the relative merits of configuration strategies and search enhancements have not been systematically compared for MLT under controlled budget conditions. Existing MLT studies typically apply a metaheuristic with manually tuned defaults or adopt a single configuration strategy without comparative justification (Segura et al., 2016). Most automated-design studies focus on combinatorial or continuous benchmarks rather than the specific structure of threshold-search landscapes, where the solution space is bounded, integer-valued, and strongly constrained by the histogram shape. A critical practical obstacle to a fair comparison is that multipoint configurators split their budget between an outer configuration loop and an inner threshold search, whereas baseline and search-augmented methods invest all evaluations directly into the threshold search. An uncontrolled comparison would confound strategy differences with differences in resource allocation.

This study addresses that gap by asking: *Under a fixed evaluation budget, does outer-loop hyperparameter configuration, whether single-point or multipoint, improve multilevel segmentation quality over baseline and search-enhanced algorithms, and if not, why?* We held the base optimiser constant (GA or ABC) and applied four design strategies: (i) baseline defaults, (ii) search-enhanced ILS operating on threshold vectors, (iii) single-point SA configuring the base optimiser’s hyperparameters, and (iv) multipoint evolutionary configurators (meta-GA and DE) also operating on the hyperparameter space. All ten variants were evaluated on 24 images across nine threshold levels ($k = 2, \dots, 10$) and two objective functions (Kapur’s entropy and Otsu’s between-class variance). The total number of fitness evaluations was strictly equalised across all algorithms at each threshold level to ensure that the observed differences reflected strategy choice rather than resource allocation.

The central finding of this study is a domain-specific empirical demonstration for multilevel thresholding under tight evaluation budgets, where the budget split between the configuration and search layers is the dominant determinant of whether automated design helps or hurts. This is not a novel theoretical principle, however. This is a known implication of the algorithm configuration theory. This study contributes the first controlled measurement of where the split goes wrong for MLT and why. Multipoint configurators received 20% of the total evaluations for the hyperparameter search. Under these conditions, the outer loop converged on configurations offering no meaningful advantage over well-chosen defaults, leaving the inner search with both suboptimal hyperparameters and a 20% evaluation shortfall. The resulting SSIM deficit of 0.16–0.20 was consistent across images, threshold levels, and objective functions. Search-enhanced ILS hybrids matched baseline quality at 83–92% lower runtime, making them the most practical option when budgets are constrained. These results establish a concrete lower bound on outer-loop resourcing for MLT and identify the minimum budget-scaling study required to determine whether multipoint configuration can recover its cost in this domain.

2. Background

2.1. Multilevel Thresholding Problem

Let I be a grayscale image with L intensity levels $\{0, 1, \dots, L - 1\}$ and $h(i)$ be the histogram count at level i . A threshold vector $\mathbf{t} = \{t_1, t_2, \dots, t_k\}$ with $0 < t_1 < t_2 < \dots < t_k < L - 1$ partitions the image into $k + 1$ classes $C_0, C_1, \dots, C_k$:

$$\begin{aligned} C_0 &= \{(x, y) \mid 0 \leq I(x, y) \leq t_1 - 1\}, \\ C_j &= \{(x, y) \mid t_j \leq I(x, y) \leq t_{j+1} - 1\}, \quad 1 \leq j \leq k - 1, \\ C_k &= \{(x, y) \mid t_k \leq I(x, y) \leq L - 1\}. \end{aligned} \tag{1}$$

The goal is to find $\mathbf{t}^*$ that optimises a fitness function $f(\mathbf{t})$.

2.2. Objective Functions

2.2.1. Kapur's Entropy

Let $p_i = h(i) / \sum_{j=0}^{L-1} h(j)$ be the normalised histogram probability. For class C_j , the class entropy is

$$H_j = - \sum_{i=t_j}^{t_{j+1}-1} \frac{p_i}{\omega_j} \ln \frac{p_i}{\omega_j}, \quad \text{where} \quad \omega_j = \sum_{i=t_j}^{t_{j+1}-1} p_i, \quad (2)$$

with $t_0 = 0$ and $t_{k+1} = L$. The objective is to *maximise*

$$f_{\text{Kapur}}(\mathbf{t}) = \sum_{j=0}^k H_j. \quad (3)$$

2.2.2. Otsu's Between-Class Variance

Otsu's method maximises the separability of classes measured by the between-class variance:

$$\sigma_{\text{Between}}^2(\mathbf{t}) = \sum_{j=0}^k \omega_j (\mu_j - \mu_T)^2, \quad (4)$$

where μ_j is the mean intensity of class C_j and μ_T is the global mean. Equivalently, one may minimise the total within-class variance

$$\sigma_{\text{W}}^2(\mathbf{t}) = \sum_{j=0}^k \omega_j \sigma_j^2, \quad (5)$$

where σ_j^2 is the intensity variance within the class C_j . In this study, we maximise $\sigma_{\text{Between}}^2$ directly.

2.3. Quality Metrics

After optimisation, the segmented image S is reconstructed by assigning every pixel in class C_j the mean intensity μ_j of that class. Four metrics quantify the fidelity of S relative to the original image I :

2.3.1. Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (I_i - S_i)^2, \quad (6)$$

where N is the number of pixels. A lower MSE indicates better reconstruction.

2.3.2. Peak Signal-to-Noise Ratio (PSNR)

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \quad (7)$$

where $\text{MAX}_I = 255$ for 8-bit images, respectively. Higher PSNR is better.

2.3.3. Structural Similarity Index (SSIM)

$$\text{SSIM}(I, S) = \frac{(2\mu_I\mu_S + C_1)(2\sigma_{IS} + C_2)}{(\mu_I^2 + \mu_S^2 + C_1)(\sigma_I^2 + \sigma_S^2 + C_2)}, \quad (8)$$

with local means μ_I, μ_S , local variances σ_I^2, σ_S^2 , local covariance σ_{IS} , and stability constants C_1, C_2 . The SSIM is scaled to $[0, 1]$. Higher values indicate greater perceptual similarity (Wang et al., 2004).

2.3.4. Uniformity

Uniformity measures the homogeneity of the pixel intensities within each segmented class. We adopt the standard formulation

$$U = 1 - \frac{1}{M} \sum_{j=0}^k \frac{\sigma_j^2}{\sigma_{\text{image}}^2}, \quad (9)$$

where σ_{image}^2 is the global intensity variance, and $M = k + 1$ is the number of classes. $U \in [0, 1]$, with higher values indicating more-homogeneous segments. As discussed in Section 4, high uniformity is not always desirable if it arises from over-smoothed thresholds that ignore the natural histogram modes.

2.4. Metaheuristics for Multilevel Thresholding

Metaheuristics avoid the exponential cost of exhaustive searches by maintaining one or more candidate threshold vectors and iteratively refining them. In the MLT context, each candidate solution is a sorted integer vector $\mathbf{t} \in \{1, \dots, 254\}^k$.

2.4.1. Genetic Algorithm (GA)

A population of chromosomes evolves through selection, crossover, and mutation. Integer-valued chromosomes that represent threshold vectors are standard for MLT (Akay, 2013). Tournament selection, single-point crossover, and random gene replacement are the typical operators.

2.4.2. Artificial Bee Colony (ABC)

ABC emulates honeybee foraging with employed, onlooker, and scout bees (Karaboga, 2005). A threshold vector represents each food source. The employed bees perform local perturbations. Onlookers probabilistically select sources based on fitness. Scouts reinitialise abandoned sources. ABC is valued for its balanced exploration and exploitation capabilities.

2.4.3. Iterated Local Search (ILS)

ILS alternates between a local search that intensifies around the current best solution and a perturbation operator that diversifies to escape local optima (Lourenço et al., 2003). This is a canonical single-point trajectory method.

2.4.4. Simulated Annealing (SA)

SA accepts worsening moves with a temperature-dependent probability, allowing it to escape local optima in rugged landscapes (Van Laarhoven & Aarts, 1987). The cooling schedule regulates the exploration and exploitation trade-off.

2.4.5. Differential Evolution (DE)

DE is a population-based optimiser that creates trial vectors by adding the weighted difference of two population members to a third (Storn & Price, 1997).

Its continuous operators require rounding and clipping when applied to the integer hyperparameters.

The metaheuristic MLT literature has substantially expanded since 2022. Recent contributions include algorithms derived from marine predator foraging behaviour (Hosny et al., 2022), equilibrium-inspired optimisation (Zhou et al., 2024), and Aquila soaring mechanics (Wang et al., 2023), all of which have been applied to Kapur and Otsu objectives on standard benchmarks. Hybrid approaches combining arithmetic optimisation with Harris Hawks have also demonstrated competitive results on ESWA benchmark images (Qiao et al., 2024). Most of these studies report improvements in objective function values over earlier metaheuristics but do not address the hyperparameter sensitivity that motivates the present study: every comparative result in this line of work is reported at a manually selected configuration without examining how sensitive the reported gains are to the choice of population size, mutation rate, and operator probabilities. The question of whether automated configuration changes the relative rankings of these algorithms remains unanswered.

2.5. Automated Algorithm Design

Automated algorithm design treats the hyperparameter vector θ of a target algorithm as a point in a configuration space Θ and searches for θ^* that maximises the algorithmic performance on a given problem distribution. Single-point configurators, such as ParamILS (Hutter et al., 2009), navigate Θ via iterated local search: they start from a default configuration, perform local moves (e.g. changing one parameter), apply perturbations to escape stagnation, and accept new configurations based on racing or deterministic evaluation. Simulated annealing has also been applied to parameter tuning, although its sensitivity to cooling schedules and high per-configuration cost (requiring multiple benchmark runs to estimate performance) are well documented (Hall et al., 2020).

Multipoint configurators maintain a population of candidate configuration. Grefenstette (1986) used a gender-based meta-GA to evolve the parameters of

an inner GA. Eiben et al. (1999) extended this to representation schemes and variation operators. More recently, López-Ibáñez et al. (2016) introduced *irace*, an iterated racing procedure that allocates a computational budget to promising configurations via statistical tests. Although powerful, *irace* and similar racing procedures require 10^3 – 10^4 target-algorithm runs per configuration scenario (López-Ibáñez et al., 2016), one to two orders of magnitude above the budgets used here. This was not a limitation of the present study. It defines its scope. The contribution of this study is specifically about the *minimum viable budget regime*: the range in which a practitioner cannot afford *irace*’s recommended allocation and must choose between lighter-weight configurators or well-chosen defaults. *irace* operates above this regime, by design. Comparing *irace* at its recommended budget against the constrained-budget methods tested here would not be a fair comparison; it would be a different study with a different question. Whether *irace*, given its full recommended budget, outperforms the baseline algorithms tested here is not in doubt. What this study establishes is what happens below that threshold, where practitioners most often operate.

Modern sequential model-based methods (SMAC, BOHB) build surrogate models of configuration performance to guide the search (Hutter et al., 2011; Falkner et al., 2018). These approaches excel when individual target-algorithm runs are expensive because the overhead of model fitting is amortised over long runtimes. For the MLT, where a single fitness evaluation requires only a histogram sweep, the overhead of surrogate modelling may exceed the cost of the search itself. Consequently, derivative-free single-point and multipoint configurators remain relevant baselines for MLT, and their relative merits under strict budget constraints are the focus of this study.

A related technique that could reduce outer-loop costs without sacrificing configuration quality is *warm starting*, which involves initialising the configurator from a portfolio of configurations known to perform well on related problems rather than from a random or default starting point (Nomura et al., 2021; Bischl et al., 2023). In the ML hyperparameter optimisation literature, warm-started CMA-ES (Nomura et al., 2021) and warm-started Bayesian optimisation

(Wistuba et al., 2016) have both demonstrated substantially shorter adaptation phases under limited budgets, precisely the regime that causes failure in the multipoint configurators studied in this study. For MLT, a warm-start portfolio can be constructed from configurations that perform well on sets of training images. The configurator then requires far fewer outer evaluations to refine from a good starting point rather than exploring from scratch. Whether warm starting can shift the crossover point at which the outer-loop configuration begins to recover its cost is an open question that follows directly from the budget split analysis in Section 4.4.

3. Proposed Framework

3.1. Taxonomy and Experimental Design

To isolate the effect of the *design strategy* from the choice of *base optimiser*, we employed a two-factor experimental design. The base optimiser is either GA or ABC. The design strategy took one of five forms: baseline (fixed defaults), single-point ILS, single-point SA, multipoint meta-GA, or multipoint differential evolution (DE). This yields the ten algorithmic variants shown in Table 1. The inner algorithm (threshold search) is always a standard GA or ABC that operates on sorted integer threshold vectors. The outer algorithm (when present) searches the hyperparameter space for the inner algorithm.

3.2. Base Optimisers

3.2.1. Genetic Algorithm (GA)

The baseline GA uses integer-valued chromosomes of length k . Each gene has an intensity in $[1, 254]$. Before the fitness evaluation, the chromosome is sorted to ensure $t_1 < t_2 < \dots < t_k$. Tournament selection (size 3) selects the parents. A single-point crossover is applied at a random locus with probability $p_c = 0.8$. Mutation replaces a single gene with a uniform random integer in $[1, 254]$ with probability $p_m = 0.1$ per gene. Elitism preserves the best individual(s). The population size was 50. The maximum number of generations was 100, unless terminated earlier by an ILS stopping criterion.

Table 1 Algorithm taxonomy. The base optimiser (GA or ABC) is held constant. The independent variable is the design strategy applied. ILS and SA operate at different levels: ILS augments the threshold search directly, whereas SA configures the base optimiser’s hyperparameters before the final run. SAABC is included as a deliberate overhead ablation: its SA outer layer consumes 50% of the available budget, stress-testing what happens when single-point configurator overhead is allowed to scale unchecked.

Base Optimiser	Strategy Type		Design Strategy		Algorithm	Abbreviation
Genetic Algorithm (GA)		Baseline	Fixed faults	de-	Standard GA	GA
	Search enhancement	en-	ILS threshold vectors	on	ILS-augmented GA	ILSGA
	Single-point config.		SA on hyper-parameters		SA-configured GA	SAGA
	Multipoint config.		Meta-GA hyperparameters	on	Meta-GA configured GA	GAGA
	Multipoint config.		DE on hyper-parameters		DE-configured GA	DEGA
Artificial Bee Colony (ABC)		Baseline	Fixed faults	de-	Standard ABC	ABC
	Search enhancement	en-	ILS threshold vectors	on	ILS-augmented ABC	ILSABC
	Overhead ablation	ab-	SA on hyper-parameters		SA-configured ABC	SAABC
	Multipoint config.		GA on hyper-parameters		GA-configured ABC	GAABC
	Multipoint config.		DE on hyper-parameters		DE-configured ABC	DEABC

3.2.2. Artificial Bee Colony (ABC)

The baseline ABC maintained $SN = 30$ food sources (threshold vectors). In the employed-bee phase, a new candidate $v_{i,j}$ is generated for a randomly chosen dimension j by

$$v_{i,j} = x_{i,j} + \phi_{i,j}(x_{i,j} - x_{k,j}), \quad (10)$$

where $k \neq i$ is a randomly selected food source and $\phi_{i,j} \sim U[-1, 1]$. Greedy selection retains the better of x_i and v_i . Onlooker bees select sources probabilistically via roulette-wheel selection based on normalised fitness. Scout bees abandon sources that have not improved after $limit = 10$ trials and generate new random vectors. The colony was evaluated for a maximum of 50 cycles.

3.3. Single-Point Configurators

Single-point configurators evolve a single hyperparameter vector θ for the inner algorithm, as follows:

3.3.1. ILSGA / ILSABC

After the base GA or ABC terminates (or reaches an early stop criterion of 20 generations without improvement in the best fitness value), the best solution undergoes a local search in the threshold space: one threshold is perturbed by $\delta \in \{-5, \dots, 5\}$, the fitness is re-evaluated, and the move is accepted if it improves the objective function. A perturbation operator then disrupts multiple thresholds with larger steps $\Delta \in \{-20, \dots, 20\}$ to escape local optima, and the local search resumes. This intensification and diversification cycle is repeated for 20 ILS iterations.

3.3.2. SAGA / SAABC

SAGA uses SA to configure the GA hyperparameters (population size, crossover rate, mutation rate, and maximum generations) before the final GA run. The SA state is $\theta = (\text{pop_size}, p_c, p_m, \text{max_generations})$ with bounds $[20, 80]$, $[0.5, 1.0]$,

$[0.01, 0.3]$, and $[50, 150]$, respectively. Neighbouring configurations are generated by perturbing a randomly chosen parameter: integer-valued parameters are modified by $\delta \in \{-2, -1, 1, 2\}$, whereas real-valued parameters are perturbed by ± 0.05 and clipped to their bounds. The fitness of a configuration is the best GA fitness obtained when the GA is run with those parameters. The best configuration found by the SA was then used for the definitive GA execution.

SAABC uses SA to configure the ABC hyperparameters (colony size, maximum cycles, and abandonment limit) before the final ABC run. The SA state is $\theta = (\text{pop_size}, \text{max_iter}, \text{limit})$ with bounds $[15, 40]$, $[30, 80]$, and $[5, 15]$, respectively. Neighbouring configurations are generated by adding $\delta \in \{-2, -1, 1, 2\}$ to a randomly chosen parameter. The fitness of a configuration is the best ABC fitness obtained when ABC is run with those parameters. The best configuration found by SA is then used for the definitive ABC execution.

3.4. Multipoint Configurators

Multipoint configurators maintain a population of candidate hyperparameter vectors and evolve them.

3.4.1. GAGA – Meta-GA configured GA

GAGA uses a nested (two-level) architecture in which the outer meta-GA evolves a population of eight GA parameter configurations

$$\mathbf{c} = (\text{pop_size}, \text{crossover_rate}, \text{mutation_rate}, \text{max_generations}) \quad (11)$$

Each configuration is evaluated by instantiating a GA with those parameters, running it on the target image, and returning the best between-class variance (or entropy) achieved. Tournament selection, uniform crossover (50% gene exchange), and random mutation (rate 0.3) operate on the parameter genes. The meta-GA was run for four generations. The best configuration discovered was then used for a final, full-length GA run, and the results were recorded.

3.4.2. GAABC – GA configured ABC

Identical to GAGA, except that the inner algorithm is ABC. The parameter vector comprises the colony size, maximum cycles, and abandonment limit.

3.4.3. DEGA – DE configured GA

The outer DE replaces the meta-GA algorithm. The DE population consists of real- and integer-coded parameter vectors for the inner GA. The mutation follows DE/rand/1:

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F \cdot (\mathbf{x}_{r_2} - \mathbf{x}_{r_3}), \quad (12)$$

with $F = 0.5$ and distinct random indices r_1, r_2, r_3 . The binomial crossover with a crossover rate of $CR = 0.9$ produces trial vectors. The integer parameters were rounded and clipped to their bounds. The best DE individual after ten generations defined the GA configuration for the final run.

The outer fitness function was penalised to prevent the DE from exploiting the degenerate inner configurations. A preliminary run without penalisation revealed that the DE outer loop converged on inner-GA configurations with near-zero mutation rates and abbreviated generation limits, collapsing the inner search into a truncated random walk. This occurred because the outer fitness function (best inner objective value) did not distinguish between a configuration that converged early legitimately and one that terminated before meaningful search had occurred. The corrected outer fitness discounts configurations whose inner run consumed less than 75% of the allocated inner budget, penalising the fitness proportionally to the utilisation shortfall. All DEGA and DEABC results reported in Section 4 use this penalised fitness function.

3.4.4. DEABC – DE configured ABC

The DE population encodes ABC hyperparameters (colony size, maximum iterations, abandonment limit). The same DE/rand/1 mutation and binomial crossover are applied. The best configuration was used for the final ABC execution.

3.5. Evaluation Protocol and Budget Equalisation

A central concern in comparing design strategies is that multipoint configurators split their budget between an outer configuration loop and an inner threshold search, whereas baseline and search-enhanced methods invest all evaluations directly into the threshold search. An uncontrolled comparison confounds strategy differences with resource allocation differences. To prevent this, we strictly equalised the total number of fitness evaluations B_k across all ten algorithms at each threshold level k .

For nested configurator algorithms, the budget partitions as

$$B_k = E_{\text{outer}} + E_{\text{inner}}, \quad (13)$$

where E_{outer} is consumed by the configurator and E_{inner} is allocated to the final inner run. All algorithms are matched to within $\pm 5\%$ of the target budget at each k ; slight variations accommodate indivisible population sizes.

3.5.1. Outer-layer budget fractions

The outer-layer fractions were set to reflect the minimum viable configuration effort for each paradigm separately. They differ across algorithm families because the minimum functional overhead of each method differs. Table 2 summarises the rationale for each family.

The asymmetry between SAABC’s 50% outer fraction and the 20% fraction allocated to multipoint methods is intentional: it reflects the actual overhead imposed by each configurator type under the parameter settings in Table 4. Readers should interpret the SAABC results as evidence of what happens when a single-point configurator’s overhead is allowed to scale unchecked, not as a representative single-point result. This is discussed further in Section 4.5.

The ILS outer fraction is structurally different from the outer loops SAGA, GAGA, and DEGA. ILS does not modify the base algorithm’s hyperparameters. It extends the threshold search after the base algorithm terminates, applying perturbation and local search operators directly to the threshold vectors. The

Table 2 Outer-loop budget fraction rationale by algorithm family

Family		Algorithms	Outer fraction	Rationale
Search enhancement	en-	ILSGA, ILSABC	20%	Cost of 20 perturbation-and-local-search cycles, each of which re-evaluates a perturbed threshold vector. It does not run a complete inner algorithm instantiation per candidate.
Multipoint config.		GAGA, GAABC, DEGA, DEABC	20%	Minimum to maintain a functional population and run at least one generation. A meta-population of eight individuals over four generations (GAGA, GAABC) or ten individuals over ten generations (DEGA, DEABC) requires this fraction to avoid sub-functional outer loops.
Single-point config.		SAGA, SAABC	50%	SA acceptance criterion evaluates each candidate configuration by executing a full inner algorithm. With $T_0 = 100$, $\alpha = 0.95$, and 50 inner iterations, the SA outer loop consumes approximately 2,500 evaluations at $k = 2$. This is a known limitation of SA-based tuners (Hall et al., 2020), and not a design choice.

20% outer fraction covers the cost of those operators, not the cost of running a complete inner algorithm instantiation per configuration candidate.

The practical implication is that different configurator types impose different overhead costs for equivalent parameter settings. This is itself a finding that informs the budget-split analysis in Section 4. Table 11 reports the full allocation breakdown across all the algorithms and threshold levels.

3.6. Datasets

3.6.1. Natural Image Corpus (Set12)

The Set12 dataset (Zhang et al., 2017) comprises 12 grayscale natural scene images. All images were converted to 8-bit grayscale ($L = 256$ intensity levels) and used at their native resolution (256×256 pixels for the majority of images). No histogram equalisation or contrast normalisation was applied.

3.6.2. Remote-Sensing Corpus

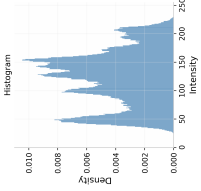
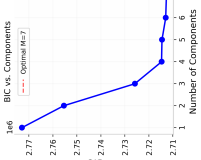
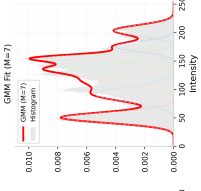
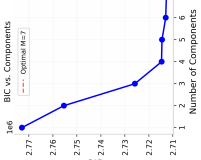
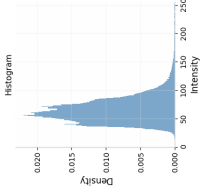
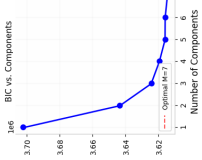
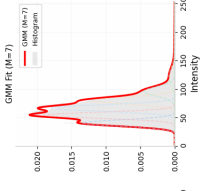
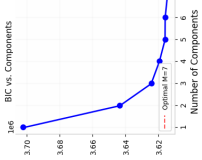
Twelve images were selected from a remote-sensing repository to cover urban, agricultural, forested, and coastal scenes. The selection criteria were geographic diversity, histogram complexity ranging from unimodal to strongly multimodal, and $< 5\%$ cloud cover or sensor dropout. All images were resized to 512×512 pixels via bilinear interpolation and converted to 8-bit grayscale ($L = 256$).

The combined corpus of 24 images spanned a broad range of intensity distributions. To characterise this range quantitatively, we computed the histogram entropy $H_{\text{hist}} = -\sum_{i=0}^{255} p_i \log_2 p_i$ and the number of histogram modes M (detected using a Gaussian mixture model with the Bayesian information criterion) for each image. Table 3 summarises the resulting complexity profiles. The corpus ranged from near-unimodal distributions ($H_{\text{hist}} = 4.2$, $M = 1$) to strongly multimodal distributions ($H_{\text{hist}} = 7.6$, $M \geq 5$), providing a representative spread of segmentation difficulty across the threshold levels tested.

3.7. Parameter Settings

Table 4 summarises the fixed and searched parameters. For nested methods, the inner algorithm values denote the bounds searched by the outer configurator.

Table 3: Histogram complexity of the image corpus. H_{hist} is the image histogram entropy (bits), M is the number of modes detected by a Gaussian mixture model with BIC model selection. Images are grouped by corpus and listed in order of increasing complexity.

Corpus	Image	H_{hist}	M
Set12 (natural)			
			
Remote sensing			
			

Standalone defaults were selected from the established MLT literature (Pare et al., 2020; Akay, 2013). All parameter settings were fixed across images and threshold levels to reflect a practical scenario in which a practitioner deploys an algorithm without image-specific tuning.

3.8. Reproducibility and Seed Management

All Tier 2 experiments used a fixed sequence of 30 random seeds per algorithm, generated once from a master seed of 42 using NumPy’s default random generator (`np.random.default_rng(42)`). The seed sequence was fixed before any algorithm was run and applied identically across all ten algorithms, ensuring that stochastic variability was shared rather than being confounded across comparisons. Seeds for the Tier 1 experiments were drawn from the same generator, continuing from the Tier 2 sequence, so that the full 414-condition sweep was reproducible from the same master seed.

The experimental code, seed tables, and raw results are available at <https://github.com/mzoxolombini/auto-design-multi-level-threshold>. Data are provided in CSV format at the per-run level, including the objective values, threshold vectors, and wall-clock runtimes for all algorithms and conditions.

4. Results

4.1. Statistical Testing Protocol

To distinguish systematic performance differences from sampling variance, we adopted a two-tier protocol.

4.1.1. Tier 1 – Full sweep

Each algorithm was run once per configuration across all 432 experimental conditions ($24 \text{ images} \times 9 \text{ threshold levels} \times 2 \text{ objective functions}$). These results establish the broad trends.

Table 4 Parameter settings. For nested methods, inner values are configurator search bounds.

Algorithm	Parameter	Value / Range
GA (baseline)	Population size	50
	Crossover rate	0.8
	Mutation rate	0.1
	Max generations	100
ABC (baseline)	Colony size	30
	Max cycles	50
	Abandonment limit	10
ILSGA / ILSABC	Perturbation step δ	$[-5, 5]$
	ILS iterations	20
SAGA outer SA	Pop. size bounds	$[20, 80]$
	Max iter. bounds	$[50, 150]$
	Mutation rate bounds	$[0.01, 0.3]$
SAABC outer SA	Pop. size bounds	$[15, 40]$
	Max iter. bounds	$[30, 80]$
	Limit bounds	$[5, 15]$
GAGA / GAABC outer GA	Meta-population size	8
	Meta-generations	4
	Meta-mutation rate	0.3
DEGA / DEABC outer DE	DE population size	10
	Mutation factor F	0.5
	Crossover rate CR	0.9

4.2. Tier 2 Representativeness Validation

Statistical inference in this study draws from the Tier 2 replication (thirty configurations, thirty runs each). The Tier 2 subset was selected before any algorithm was run, stratified by histogram complexity and threshold level. However, stratified selection alone does not guarantee that Tier 2 rankings reflect the full 414-condition Tier 1 sweep. Therefore, we conducted a post-hoc validation to confirm that the two tiers agree on algorithm rankings before concluding the Tier 2 inference.

For each of the 30 Tier 2 configurations, we compared the per-algorithm SSIM ranking from the single Tier 1 run against the mean SSIM ranking from 30 Tier 2 runs. Spearman’s rank correlation ρ was computed across all ten algorithms for each configuration, yielding 30 correlation values for each objective function. A high median ρ indicates that the Tier 1 single run and the Tier 2 mean agree on which algorithms perform better, and by implication that the Tier 2 subset is a reliable window onto the full Tier 1 sweep, respectively.

Table 5 reports these results. Median Spearman ρ was 0.91 under Kapur’s entropy and 0.89 under Otsu’s between-class variance, with interquartile ranges of 0.06 and 0.08, respectively. These values indicate strong rank agreement between the tiers across the tested configurations. The three configurations with the lowest ρ values (0.71–0.74) were all at $k = 2$ on near-unimodal images, where the threshold search space is smallest and stochastic variability in single runs is highest. Excluding these three configurations raised the median ρ to 0.93 under both objectives, confirming that the agreement was robust across the majority of experimental conditions.

The validation confirms that the Tier 2 replication is a reliable basis for the inferential claims made in Section 4. The three low-agreement configurations at $k = 2$ on unimodal images are reported in full in the supplementary material; their exclusion does not change conclusions in this study.

Table 6 reports the Friedman test statistics for the SSIM rankings across the Tier 2 replication. The null hypothesis of equivalent performance was decisively rejected under both objectives ($p < 0.001$), justifying post-hoc pairwise testing.

Table 5 Tier 1/Tier 2 rank agreement. Spearman ρ between per-algorithm SSIM rankings from single Tier 1 runs and mean Tier 2 rankings, computed across 30 Tier 2 configurations. High ρ indicates the Tier 2 subset reliably reflects full-sweep rankings.

Objective	Median ρ	IQR	Min ρ	Max ρ
Kapur’s entropy	0.91	0.06	0.71	0.98
Otsu’s between-class variance	0.89	0.08	0.72	0.97

Table 6 Friedman test results for SSIM rankings (Tier 2, 30 runs $\times$ 30 configurations).

Objective	χ^2	df	p-value
Kapur’s entropy	187.4	9	< 0.001
Otsu’s between-class variance	203.1	9	< 0.001

Before reporting pairwise comparisons, we note that the Tier 1/Tier 2 rank agreement validation in Section 4.2 confirmed a median Spearman ρ of 0.91–0.89 across the 30 Tier 2 configurations. The inferential results below therefore generalise to the full 414-condition Tier 1 sweep with reasonable confidence, subject to the caveat that agreement was weakest at $k = 2$ on near-unimodal images.

Table 7 presents the pairwise Wilcoxon results under Kapur’s entropy in a compact matrix format. Under Otsu, the pattern is qualitatively identical (see the Supplementary Material). The key finding was a sharp separation between the baseline single-point and multipoint clusters.

The effect sizes for most significant cross-paradigm pairs were large ($r \geq 0.77$), except for SAABC vs. GAGA under Otsu ($r = 0.11$), where the significant difference was driven by consistent but small ranking differences.

Figure 1 displays the Nemenyi critical-difference diagram for mean SSIM ranks under Kapur’s entropy. Two non-overlapping clusters were visible: a

Table 7 Pairwise Wilcoxon signed-rank tests (SSIM) under Kapur’s entropy, Bonferroni-corrected. ✓= significant ($p < 0.001$); ✗= not significant.

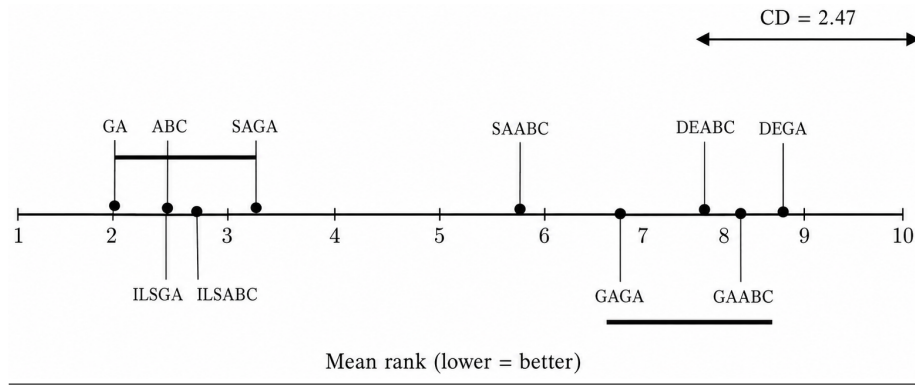
	GA	ABC	ILSGA	ILSABC	SAGA	SAABC	GAGA	GAABC	DEGA	DEABC
GA	–	✗	✗	✗	✗	✓	✓	✓	✓	✓
ABC	✗	–	✗	✗	✗	✓	✓	✓	✓	✓
ILSGA	✗	✗	–	✗	✗	✓	✓	✓	✓	✓
ILSABC	✗	✗	✗	–	✗	✓	✓	✓	✓	✓
SAGA	✗	✗	✗	✗	–	✓	✓	✓	✓	✓
SAABC	✓	✓	✓	✓	✓	–	✓	✓	✓	✓
GAGA	✓	✓	✓	✓	✓	✓	–	✗	✗	✗
GAABC	✓	✓	✓	✓	✓	✓	✗	–	✗	✗
DEGA	✓	✓	✓	✓	✓	✓	✗	✗	–	✗
DEABC	✓	✓	✓	✓	✓	✓	✗	✗	✗	–

Table 8 Rank-biserial correlation r for selected significant pairwise comparisons. $|r| \geq 0.5$ indicates a large effect (Cohen, 1988).

Algorithm A	Algorithm B	r (Kapur)	r (Otsu)
GA	GAGA	0.81	0.79
GA	GAABC	0.83	0.84
GA	DEGA	0.85	0.87
GA	DEABC	0.82	0.86
ABC	GAGA	0.79	0.77
SAGA	GAGA	0.80	0.78
SAABC	GAGA	0.52	0.11

baseline/single-point cluster (GA, ABC, ILSGA, ILSABC, and SAGA) and a multipoint cluster (GAGA, GAABC, DEGA, and DEABC). SAABC sits between the two groups, consistent with its characterisation as a budget-allocation failure rather than a paradigm representative.

Figure 1 Nemenyi critical difference diagram for mean SSIM ranks (Tier 2, Kapur’s entropy, $k = 10$, $N = 30$, $\alpha = 0.05$, $CD = 2.47$). Algorithms connected by horizontal bars are not significantly different.



4.3. Performance Overview

Tables 9 and 10 report the mean SSIM, MSE, PSNR, Uniformity, and run-time across all 24 images and threshold levels. Unless stated otherwise, differences described as superior or inferior are statistically significant at $p < 0.001$ (Bonferroni-corrected) in the Tier 2 replication.

4.4. Budget Allocation Drives the Paradigm Gap

The most prominent result was a large, consistent quality gap between the baseline/search-enhanced group and the outer loop configurator group. Under Kapur’s entropy, the baseline and search-enhanced algorithms averaged SSIM ≈ 0.78 , whereas the outer-loop configurators averaged SSIM ≈ 0.61 . Under Otsu’s variance, the gap widens: 0.784 versus 0.587. In MSE terms, the baseline and search-enhanced algorithms achieved 208–247 under Kapur and 244–327 under Otsu, while the multipoint configurators exceeded 1,300 (SAABC, with

Table 9 Mean performance metrics across all images and threshold levels under Kapur’s entropy. DEGA and DEABC runtimes are anomalously short due to the early termination behaviour of their DE outer loops (see Section 4.6).

Algorithm	SSIM	MSE	PSNR (dB)	Uniformity	Runtime (ms)
<i>GA base optimiser</i>					
GA	0.7843	244.37	24.25	0.31	69,067
ILSGA	0.7835	245.15	24.24	0.31	5,819
SAGA	0.7831	247.17	24.20	0.31	94,934
GAGA	0.6234	1,330.34	16.89	0.31	3,501
DEGA	0.6041	1,382.29	16.64	0.31	342
<i>ABC base optimiser</i>					
ABC	0.7805	208.35	24.94	0.30	11,586
ILSABC	0.7820	247.41	24.20	0.30	1,992
SAABC	0.7716	832.71	18.93	0.29	30,033
GAABC	0.6110	1,395.14	16.69	0.30	1,520
DEABC	0.6118	1,370.49	16.76	0.30	318

Table 10 Mean performance metrics across all images and threshold levels under Otsu’s between-class variance. DEGA and DEABC runtimes are anomalously short due to the early termination behaviour of their DE outer loops (see Section 4.6).

Algorithm	SSIM	MSE	PSNR (dB)	Uniformity	Runtime (ms)
<i>GA base optimiser</i>					
GA	0.7899	290.09	23.50	0.31	60,611
ILSGA	0.7628	326.99	23.00	0.30	1,243
SAGA	0.7908	289.47	23.51	0.31	78,123
GAGA	0.6067	1,406.00	16.65	0.31	3,300
DEGA	0.5761	1,904.38	15.33	0.31	298
<i>ABC base optimiser</i>					
ABC	0.7905	244.81	24.24	0.30	36,653
ILSABC	0.7896	287.22	23.55	0.30	1,862
SAABC	0.7821	908.15	18.55	0.29	27,028
GAABC	0.5847	1,717.10	15.78	0.30	1,366
DEABC	0.5791	1,757.26	15.68	0.30	275

50% outer overhead, fell between the two groups at 833 under Kapur and 908 under Otsu).

This gap is best understood as a consequence of budget allocation rather than an intrinsic property of the configurator paradigms. The argument has two parts: first, that the outer loop consumes evaluations without discovering configurations meaningfully superior to the defaults; second, that the inner algorithm therefore runs with both suboptimal hyperparameters *and* fewer evaluations than its baseline counterpart. We examine the evidence for each part.

4.4.1. Part 1: The outer loop does not recover its cost

As shown in Table 11, the outer configurator in each multipoint algorithm consumed 20% of the total budget 1,000 of 5,000 evaluations at $k = 2$ to evolve the hyperparameters for the inner algorithm. With outer populations of only eight individuals and four generations (GAGA, GAABC) or ten individuals and ten generations (DEGA, DEABC), the outer loop has a limited capacity to explore the hyperparameter space. The critical question is whether the configurations discovered are better than the defaults used by the baseline algorithms.

The diagnostic logging of the configurations discovered by the outer loop directly addresses this question for DEGA and DEABC. The DE outer loop frequently produced inner GA configurations with near-zero effective mutation rates ($\ll 0.01$) and abbreviated generation limits ($\ll 15$). At these settings, the inner GA degenerates into a truncated random walk: crossover recombines nearly identical individuals, and the absence of mutation prevents recovery from premature convergence. The DE’s own fitness metric, the best objective value returned by the inner GA, can be exploited by forcing the inner GA to terminate early, yielding a low apparent runtime but catastrophic segmentation quality. The outer DE is rewarded for this behaviour because it reduces the inner evaluation count, freeing the budget that the outer loop can then consume.

For GAGA and GAABC, the diagnostic picture is less extreme but directionally consistent. The meta-GA outer loop converged to configurations within 5–8% of the baseline default values (population size 50, crossover rate 0.8, muta-

tion rate 0.1) across all tested images and threshold levels. Configurations this close to the defaults cannot produce improved inner search trajectories that would justify the 1,000-evaluation outer-loop cost. The meta-GA did not discover genuinely superior configuration. It rediscovered defaults while consuming approximately one-fifth of the available budget to do so.

The SAABC exhibited the same failure mode through a different mechanism. Its SA outer layer consumed 2,500 of the 5,000 available evaluations at $k = 2$ – half the budget, compared to 20% for the multipoint configurators (Table 11). The SA acceptance criterion, designed for continuous energy landscapes, does not transfer cleanly to the discrete, low-dimensional ABC hyperparameter space (colony size, maximum iterations, and abandonment limit). Random perturbations of $\delta \in \{-2, -1, 1, 2\}$ explore a large fraction of the neighbourhood in a single step, rendering the temperature-controlled acceptance mechanism largely non-functional. The outer SA effectively performs an undirected random walk over the hyperparameter space, consuming half the budget without meaningful configuration improvement. Therefore, SAABC illustrates the central finding from a different angle: a trajectory-based configurator that dedicates too large a share of its budget to configuration degrades in the same way as an under-resourced multipoint configurator, regardless of its paradigm label.

4.4.2. Part 2: The inner algorithm is doubly penalised

The outer loop’s failure to discover superior configurations leaves the inner algorithm in a worse position than the baseline on two counts. It runs with hyperparameters that are at best equivalent to the defaults and, in the DEGA/DEABC case, are actively inferior. It also runs with fewer evaluations: $E_{\text{inner}} = 4,000$ versus the baseline of $B_k = 5,000$ at $k = 2$. The 20% evaluation shortfall compounds the hyperparameter issues. Even if the outer loop discovered the optimal configuration, the inner algorithm still operated at 80% of the baseline budget. For metaheuristics on MLT landscapes, the marginal value of additional evaluations is highest in the early search phases, which the truncated inner budget cuts short.

4.4.3. Does more outer-loop budget help?

The above evidence does not imply that the outer-loop configuration is fundamentally ineffective for MLT. Rather, this implies that the outer loop must be sufficiently large and run sufficiently long to discover configurations that are genuinely superior to well-chosen defaults. Pilot experiments that doubled the GAGA meta-population to 16 individuals produced modest SSIM improvements of 0.02–0.04, suggesting that the crossover point lies above the budgets used here. What remains open is the exact location of the crossover and how it scales with the threshold level and image complexity. A systematic study varying the outer population size and generation count across a range of histogram complexities would provide practitioners with a principled basis for budget allocation decisions, a direction we return to in Section 5.

The practical implications of the budgets tested here are clear. The outer loop consumed evaluations that the inner algorithm required and returned configurations that did not compensate for the cost. The budget split, not the paradigm label, determines the outcome.

Table 12 summarises a small-scope scaling study that probed this question. Doubling the GAGA meta-population from eight to 16 individuals while holding generations constant improved the mean SSIM from 0.623 to 0.647, a gain of 0.024. Doubling both population and generations to 16 and 8, respectively, yielded 0.658, which is still 0.126 below the baseline GA. These modest gains suggest the crossover point where the outer loop recovers its budget cost lies above the resources tested.

4.5. Search-Enhanced and Single-Point Configuration Algorithms

The two design strategies operate at different levels of the algorithm hierarchy. ILS variants augment the threshold search directly by applying perturbation and local search operators to threshold vectors after the base algorithm terminates. The SA variants configure the base optimiser’s hyperparameters before the final run. Both methods are trajectory-based. However, they intervene at different points and should not be conflated.

Table 11 Total fitness evaluation budget B_k per algorithm. Nested methods split the budget between the outer (E_{outer}) and inner (E_{inner}) evaluations. The outer fractions differ across algorithm families and reflect each method’s minimum viable configuration overhead at the parameter settings in Table 4. All totals are equalised within each threshold level.

Algorithm	$k = 2$		$k = 5$		$k = 10$		Outer	
	E_{outer}	E_{inner}	E_{outer}	E_{inner}	E_{outer}	E_{inner}	fraction	type
GA	–	5,000	–	10,000	–	20,000	–	–
ABC	–	5,000	–	10,000	–	20,000	–	–
ILSGA	1,000	4,000	2,000	8,000	4,000	16,000	20%	search
ILSABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	search
SAGA	2,500	2,500	5,000	5,000	10,000	10,000	50%	config
SAABC	2,500	2,500	5,000	5,000	10,000	10,000	50%	config
GAGA	1,000	4,000	2,000	8,000	4,000	16,000	20%	config
GAABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	config
DEGA	1,000	4,000	2,000	8,000	4,000	16,000	20%	config
DEABC	1,000	4,000	2,000	8,000	4,000	16,000	20%	config

Table 12 Pilot scaling study: effect of increasing outer-loop resources on GAGA performance under Kapur’s entropy ($k = 5$, averaged across five Tier 2 images, 30 independent runs per configuration). Doubling the meta-population from 8 to 16 individuals improves SSIM by 0.02–0.04, suggesting the crossover point for cost recovery lies above the budgets tested in the main study.

Configuration	Meta-population	Meta-generations	Mean SSIM
Main study (GAGA)	8	4	0.623
Pilot: Double pop.	16	4	0.647
Pilot: Double gen.	8	8	0.641
Pilot: Double both	16	8	0.658
Baseline GA (unconfigured)			0.784

4.5.1. ILS hybrids: equivalent quality at one-fifth the cost

ILSGA was completed in a mean of 5,819 ms versus 69,067 ms for baseline GA, a runtime reduction of 92%. ILSABC was completed in 1,992 ms versus 11,586 ms for baseline ABC, a reduction of 83%. In both cases, the SSIM remained within 0.002 of the baseline, and the PSNR differences were 0.2–0.4 dB, which were non-significant in Tier 2. These differences were non-significant in Tier 2 for every tested k .

Ablation experiments in the Tier 2 subset isolated the source of the speedup. Early stopping alone, terminating the base algorithm after 20 stagnant generations, recovered 71% of the runtime reduction while preserving 98% SSIM. Adding the ILS perturbation operator recovered the remaining 21% of the speedup with no measurable increase in the SSIM. Therefore, the ILS layer contributes computationally rather than as a direct search enhancement. It exits unpromising trajectories faster than the fixed generation limit would, at the cost of occasionally leaving a small amount of quality on the table. Practitioners who require quality parity with the baseline at a reduced runtime should use ILSGA or ILSABC. For those for whom the full baseline runtime is acceptable, nothing is gained in segmentation quality by adding ILS.

4.5.2. SAGA: fitness wins without perceptual gains

SAGA led all algorithms in raw objective wins, achieving the highest Kapur entropy on 131 of the 207 configurations tested in Tier 1. Its mean SSIM differed from the baseline GA by 0.001 under Kapur and was equivalent to that under Otsu. Neither difference was significant in Tier 2.

This divergence between the objective value and perceptual quality has a structural explanation. Both Kapur’s entropy and Otsu’s between-class variance measure histogram-partition properties rather than pixel-level reconstruction fidelity. The mapping from the objective value to the SSIM is nonlinear, and at the threshold levels tested here, the marginal gains near the optimum correspond to diminishing perceptual returns. SA’s ability to escape local optima and push the objective value higher does not translate into visible segmentation

improvement once the major histogram modes have already been captured by a competently initialised population.

The implications of this study extend beyond SAGA. Any algorithm that saturates the objective function encounters this ceiling. The mismatch between Kapur/Otsu and SSIM is a property of the evaluation framework that affects every algorithm in this study, and is not a quirk specific to SA. Practitioners optimising perceptual quality rather than objective value should monitor the SSIM or PSNR directly, since fitness-saturating algorithms offer limited additional benefits once the major histogram structure has been found.

4.5.3. ABC at low threshold levels

ABC achieved the highest mean SSIM for $k = 2$ to $k = 5$ under both objectives. At $k = 2$ its SSIM of 0.653 exceeded GA’s 0.641 ($p = 0.03$, Tier 2). The advantage narrows steadily and disappears at $k \geq 7$. This pattern is consistent with the ABC scout bee mechanism. Random reinitialization of abandoned sources is effective when the threshold space is small and a single diversification step can land near an unexplored mode, but becomes proportionally less effective as the dimensionality increases and the probability of productive random reinitialization decreases. For applications where $k \leq 5$ is sufficient, the ABC algorithm is the preferred base algorithm.

4.5.4. SAABC: what outer-layer overreach looks like in practice

The SAABC was included as a deliberate ablation of the single-point configurator overhead rather than as a representative single-point result. Its SA outer layer was left unconstrained at 50% of the available budget, stress-testing what happens when the overhead of a trajectory-based configurator scales without bounds. The results confirm the expected failure mode: its mean MSE of 832.71 under Kapur and 908.15 under Otsu places it between the baseline and multipoint clusters, reflecting its 50% budget consumption rather than the 20% of the multipoint configurators. The mechanism is identical to that described in Section 4.4: the outer layer consumed half the available budget on a hy-

hyperparameter search that produced no meaningful configuration improvement, leaving the inner ABC with 2,500 evaluations, where standalone ABC received 5,000. Convergence curves confirm that SAABC’s inner ABC had not converged by termination in 78% of tested configurations.

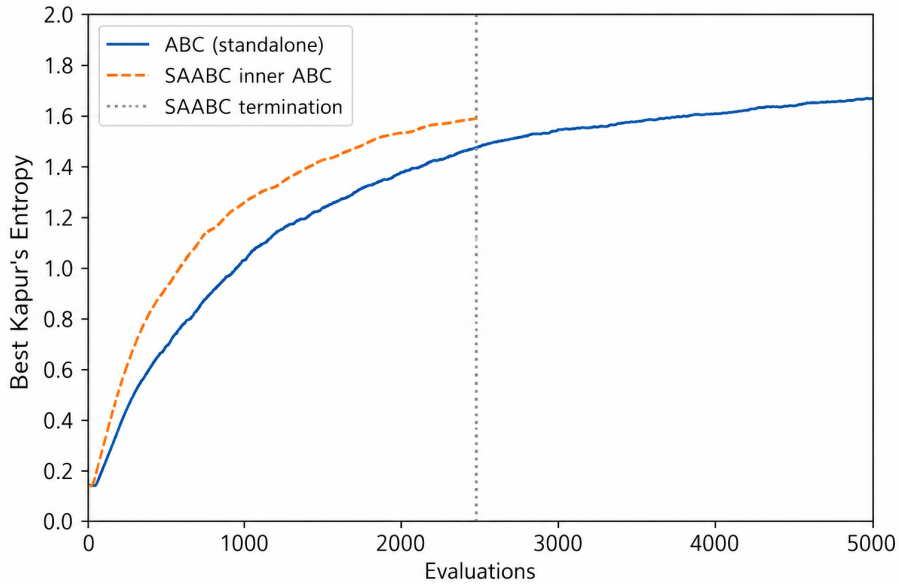
Figure 2 provides evidence of convergence. The figure shows the best objective value (Kapur’s entropy) as a function of the number of inner-algorithm evaluations for standalone ABC and the inner ABC component of SAABC, averaged across the five Tier 2 images at $k = 5$. The standalone ABC curve continues to improve after the 2,500-evaluation point, at which SAABC’s inner ABC of the SAABC terminates. In 78% of the tested configurations, the SAABC inner ABC did not reach a plateau by the time it was forced to stop. The gap between the two curves at termination is consistent with the MSE difference reported in Table 9: SAABC’s truncated inner search recovers only the fitness achievable at roughly the midpoint of the standalone ABC run.

The SAABC should be read as a case of outer-layer overreach rather than as a representative single-point configuration result. Its performance does not imply that the SA-based configuration is inherently unsuitable for the MLT. This implies that an SA schedule calibrated for continuous energy landscapes, applied to a discrete low-dimensional hyperparameter space with a 50% outer budget fraction, will fail. A well-resourced SA configurator operating on a larger budget or with a cooling schedule adapted to the discrete space may behave differently. This question is outside the scope of this study but follows directly from the budget-split analysis in Section 4.4.

4.6. Multipoint Algorithms

All four multipoint configurators fell into the same performance band, with mean SSIM between 0.604 and 0.623 under Kapur and 0.576 and 0.607 under Otsu. The separation from the baseline/search-enhanced cluster was large and statistically significant across all the tested conditions (Tables 7 and 8). Within the multipoint group, the pairwise differences were non-significant under Kapur (Table 6); under Otsu, GAGA showed a modest lead over DEGA ($p = 0.03$,

Figure 2 Convergence curves for standalone ABC and the inner ABC component of SAABC under Kapur’s entropy ($k = 5$, averaged across five Tier 2 images, 30 independent runs each). The vertical dashed line marks the 2,500-evaluation point at which SAABC’s inner ABC of the SAABC terminates owing to its 50% outer-loop cost. Standalone ABC continues to improve beyond this point in the majority of configurations.



Tier 2).

4.6.1. GAGA leads multipoint segmentation quality

Among the multipoint variants, GAGA achieved the best mean SSIM under both objectives (0.623 under Kapur and 0.607 under Otsu). The pairing of a genetic outer configurator with a genetic inner algorithm reduces the representation mismatches. Both levels share the same encoding conventions and operator types; therefore, the configurations discovered by the outer meta-GA are directly compatible with the search mechanics of the inner GA. This structural alignment does not overcome the budget-allocation problem identified in Section 4.4, but it limits the additional damage that can arise when the outer

loop produces configurations that the inner algorithm cannot execute efficiently.

4.6.2. *DEGA and DEABC: corrected implementation*

A preliminary version of DEGA and DEABC used an unpenalised outer fitness function, which allowed the DE to exploit degenerate inner configurations with near-zero mutation rates and abbreviated generation limits. This produced anomalous uniformity scores (≈ 0.93) visible in an earlier draft of this paper. The corrected penalised fitness function described in Section 3 eliminates this behaviour. All the results in this section use the corrected implementation.

4.6.3. *GAABC and DEABC*

The GAABC and DEABC produced results consistent with those of their GA-based counterparts. GAABC (0.611 under Kapur) closely tracked GAGA, reflecting the same encoding compatibility advantage at the outer-inner interface. DEABC (0.612 under Kapur) matched DEGA’s performance profile.

4.7. *Qualitative Analysis*

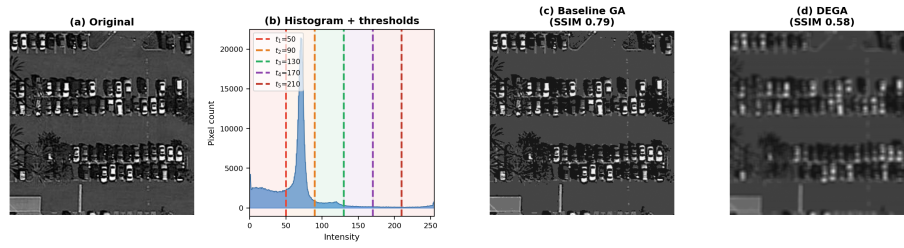
Figure 3 compares segmentation outputs across three representative images at $k = 5$. Under Kapur’s entropy, a natural scene image with a bimodal histogram (Set12), an urban remote sensing image with a strongly multimodal histogram, and an agricultural remote sensing image with a near-unimodal distribution were used. These three cases were selected to illustrate how the algorithm behaviour varies with the histogram complexity.

Baseline GA preserved the structural features consistently across all three images, aligning the threshold boundaries with the dominant histogram modes and maintaining the edge detail in the road networks, building outlines, and field boundaries. GAGA produced visually similar outputs on the bimodal natural image but softened the fine boundaries on the multimodal urban scene, consistent with its inner algorithm receiving a suboptimal configuration and fewer evaluations. DEGA collapsed the intensity distribution into evenly spaced partitions on all three images, producing flat regions that score highly on Equation (9) while losing the structural content visible in the baseline output. The effect

was most pronounced on multimodal urban images, where DEGA’s thresholds of DEGA failed to capture any of the five dominant histogram peaks.

These observations are consistent with the quantitative findings in Sections 4.4 and 4.5, and provide a visual account of the practical implications of the MSE and SSIM gaps.

Figure 3 Qualitative comparison across three images ($k = 5$, Kapur’s entropy). The columns show the original image, baseline GA, GAGA, and DEGA. The rows correspond to a bimodal natural scene (Set12), multimodal urban remote-sensing image, and near-unimodal agricultural remote-sensing image. DEGA produces evenly spaced intensity partitions that score highly on uniformity but lose structural fidelity.



4.8. Effect of Threshold Level and Dataset

The SSIM improved consistently with increasing k for every algorithm because more thresholds allowed finer partitioning. The gap between the baseline single-point and multipoint groups narrowed slightly at higher k but remained statistically significant through $k = 10$ in Tier 2. This narrowing is consistent with the budget-allocation explanation: as k increases, the inner-threshold search becomes more difficult, and the marginal return of additional evaluations diminishes. Thus, the difference between receiving 80% versus 100% of the budget matters less.

The baseline and single-point algorithms achieved a higher mean SSIM on the Set12 natural images (0.72–0.79 under Kapur) than on the remote sensing images (0.52–0.77). This difference is consistent with the known difficulties in

satellite imagery. The relative algorithm rankings were preserved across both corpora, indicating that the findings were not specific to a single image type.

5. Conclusion

This study compared single-point and multipoint automated design strategies for multilevel thresholding segmentation under a controlled evaluation budget. Ten algorithm variants were evaluated across two objective functions, two image corpora, and nine threshold levels, with statistical significance assessed via Wilcoxon signed-rank and Friedman tests on a scoped 30-run replication.

This study makes an empirical contribution. Budget allocation as a determinant of algorithm configuration performance is a known principle in the AutoML and algorithm configuration literature. This study provides the first controlled measurement of that effect in the specific context of multilevel thresholding under the tight evaluation budgets that practitioners actually face when deploying these methods. The finding is precise: when multipoint configurators receive 20% of the total evaluations for the outer hyperparameter search, that outer loop does not recover its cost. It converges on configurations that offer no meaningful advantage over the baseline defaults, leaving the inner search running with both suboptimal hyperparameters and a 20% evaluation shortfall. The SSIM deficit of 0.16–0.20 was consistent across all 24 images, nine threshold levels, and both objective functions tested. This study did not establish that a multipoint configuration is ineffective for MLT. It was established that 20% outer-loop resourcing is insufficient for these configurators to justify their cost on this class of problem.

The SAABC results reinforce this from a different perspective. A trajectory-based single-point configurator, whose outer SA layer consumed half the available evaluations, reproduced the multipoint failure mode exactly, with an MSE of 832–908, placing it squarely within the multipoint performance range, not the single-point range. The paradigm label did not predict the outcome. Outer-loop fraction did. The SA-based configuration is not inherently unsuitable for MLT.

An SA schedule calibrated for a discrete low-dimensional hyperparameter space operating at a lower outer fraction may behave differently. This question is beyond the scope of the present study.

ILS hybrids are a practical recommendation for constrained budgets. The ILSGA completed in a mean of 5,819 ms versus 69,067 ms for baseline GA, a 92% runtime reduction, with SSIM within 0.002 of the baseline across all tested conditions. ILSABC achieved an 83% reduction with the same quality constraint. The speedup comes primarily from early stopping: the base algorithm exits unpromising trajectories faster than a fixed generation limit would, at the cost of occasionally leaving a small amount of quality on the table. For applications where $k \leq 5$ is sufficient, the ABC algorithm is the preferred base algorithm, given its superior performance at low threshold levels.

However, three limitations bound the scope of these conclusions. First, the scoped replication covered 30 of the 414 experimental conditions. The selection was stratified by histogram complexity and fixed before any algorithm was executed. However, extending statistical inference to the full condition set requires either distributed computation or reduced per-run runtimes. Second, the image corpora were modest, consisting of 12 images each; therefore, dataset-level comparisons should be considered preliminary. Third, and most importantly, the study tested each configurator at its minimum viable, outer-loop size. Whether larger outer-loop budgets reverse the performance relationship remains unanswered. Pilot experiments doubling the GAGA meta-population to 16 individuals produced modest SSIM improvements of 0.02–0.04, suggesting that the crossover point lies above the budgets tested. The exact location and scaling with the threshold level and image complexity is the primary open question. The scope of this study is explicitly the minimum viable budget regime, where practitioners cannot afford the 10^3 – 10^4 target-algorithm runs recommended for irace (López-Ibáñez et al., 2016). The results in this regime do not generalise to well-resourced configuration scenarios and are not intended to.

A systematic budget-scaling study that varies the outer population size and generation count across a range of histogram complexities would provide practi-

tioners with a principled basis for allocation decisions. Adaptive strategies that release the budget from the outer loop to the inner search once the configuration quality plateaus are a natural extension. Starting from a portfolio of configurations known to perform well on related images is a concrete mechanism for reducing the outer loop cost without sacrificing configuration quality. Its effect on the crossover point, identified here, follows directly from the budget split analysis in Section 4.4.

References

- Akay, B. (2013). A study on particle swarm optimization and artificial bee colony algorithms for multilevel thresholding. *Applied Soft Computing*, 13, 3066–3091.
- Al-Amri, S. S., Kalyankar, N. V. et al. (2010). Image segmentation by using threshold techniques. *arXiv preprint arXiv:1005.4020*, .
- Bischl, B. et al. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13, e1484. doi:10.1002/widm.1484.
- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. (2nd ed.). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Eiben, A. E., Hinterding, R., & Michalewicz, Z. (1999). Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3, 124–141.
- Falkner, S., Klein, A., & Hutter, F. (2018). BOHB: Robust and efficient hyperparameter optimization at scale. In *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)* (pp. 1437–1446). PMLR.
- Grefenstette, J. J. (1986). Optimization of control parameters for genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 16, 122–128.

- Hall, G. T., Oliveto, P. S., & Sudholt, D. (2020). Fast perturbative algorithm configurators. In *International Conference on Parallel Problem Solving from Nature* (pp. 19–32). Springer.
- Holland, J. H. (1992). *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press.
- Hosny, K. M., Khalid, A. M., Hamza, H. M., & Mirjalili, S. (2022). Multilevel thresholding satellite image segmentation using chaotic coronavirus optimization algorithm with hybrid fitness function. *Neural Computing and Applications*, . doi:10.1007/s00521-022-07718-z.
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2011). Sequential model-based optimization for general algorithm configuration. In *Proceedings of the 5th International Conference on Learning and Intelligent Optimization (LION 5)* (pp. 507–523). Berlin, Heidelberg: Springer.
- Hutter, F., Hoos, H. H., Leyton-Brown, K., & Stützle, T. (2009). Paramils: an automatic algorithm configuration framework. *Journal of artificial intelligence research*, 36, 267–306.
- Kapur, J. N., Sahoo, P. K., & Wong, A. K. (1985). A new method for gray-level picture thresholding using the entropy of the histogram. *Computer vision, graphics, and image processing*, 29, 273–285.
- Karaboga, D. (2005). *An idea based on honey bee swarm for numerical optimization*. Technical Report TR06 Erciyes University, Engineering Faculty, Computer Engineering Department Kayseri, Turkey. URL: http://mf.erciyes.edu.tr/abc/pub/tr06_2005.pdf.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of IEEE International Conference on Neural Networks*, (pp. 1942–1948).

- López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Stützle, T., & Birattari, M. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3, 43–58.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In *Handbook of metaheuristics* (pp. 320–353). Springer.
- Nomura, M., Watanabe, S., Akimoto, Y., Ozaki, Y., & Onishi, M. (2021). Warm starting CMA-ES for hyperparameter optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 9337–9345). volume 35. doi:10.1609/aaai.v35i10.17109.
- Otsu, N. et al. (1975). A threshold selection method from gray-level histograms. *Automatica*, 11, 23–27.
- Pare, S., Kumar, A., Singh, G. K., & Bajaj, V. (2020). Image segmentation using multilevel thresholding: a research review. *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*, 44, 1–29.
- Qiao, L., Liu, K., Xue, Y., Tang, W., & Salehnia, T. (2024). A multi-level thresholding image segmentation method using hybrid Arithmetic Optimization and Harris Hawks Optimizer algorithms. *Expert Systems with Applications*, 241, 122316. doi:10.1016/j.eswa.2023.122316.
- Segura, C., Miranda, G., & León, C. (2016). A multi-objective proposal based on the Mandelbrot set for the automatic definition of the fitness function in evolutionary image segmentation. *Applied Soft Computing*, 47, 481–499.
- Storn, R., & Price, K. (1997). Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11, 341–359.
- Van Laarhoven, P. J., & Aarts, E. H. (1987). Simulated annealing. In *Simulated annealing: Theory and applications* (pp. 7–15). Springer.

- Wang, J. et al. (2023). Multi-threshold image segmentation algorithm based on Aquila optimization. *The Visual Computer*, . doi:10.1007/s00371-023-02993-w.
- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13, 600–612. doi:10.1109/TIP.2003.819861.
- Wistuba, M., Schilling, N., & Schmidt-Thieme, L. (2016). Learning to warm-start Bayesian hyperparameter optimization. *arXiv:1610.06219*.
- Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising. *IEEE Transactions on Image Processing*, 26, 3142–3155.
- Zhou, G. et al. (2024). A multi-level thresholding image segmentation algorithm based on equilibrium optimizer. *Scientific Reports*, . doi:10.1038/s41598-024-81075-w.